Dynamic Programming And NP

Dynamic Programming

is a technique in computer programming that helps to efficiently solve a class of problems that have overlapping sub-problems and optimal substructure property.

like the divide-and-conquer method, solves problems by combining the solutions to sub-problems.

dynamic programming applies when the sub-problems overlap that is, when sub-problems share sub-sub-problems.

Matrix-chain multiplication

We are given a sequence (chain) (A1 ; A2 ; ... ; An) of n matrices to be multiplied

A product of matrices is fully parenthesized if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses. For example, if the chain of matrices is (A1 ; A2 ; A3 ; A4), then we can fully parenthesize the product A1 A2 A3 A4 in five distinct ways:

- (A1*(A2*(A3*A4)))
- (A1*((A2*A3)*A4))
- ((A1*A2)*(A3*A4))
- (((A1*A2)*A3*)A4)

We can multiply two matrices A and B only if they are compatible: the number of columns of A must equal the number of rows of B.

To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain (A1 ; A2 ; A3)

A1 = 10*100 dimensions

A2 = 100*5 dimensions

A3 = 5*50 dimensions

((A1*A2)*A3) = (10*100*5)+(10*5*50)= 7500 scalar multiplications

(A1*(A2*A3)) = (100*5*50) + (10*100*50) = 75000 scalar multiplications

computing the product according to the first parenthesization is 10 times faster.

NP-Completeness

polynomial-time algorithms: on inputs of size n, their worst-case running time is O(n^k) for some constant k.

No polynomial-time algorithm has yet been discovered for an NP-complete problem, nor has anyone yet been able to prove that no polynomial-time algorithm can exist for any one of them.

Type of NP

- NP
- NP-Complete
  - longest simple paths
  - Traveling Salesman
  - Hamiltonian path
- NP-Hard

Decision problems vs. optimization problems

Many problems of interest are optimization problems, in which each feasible solution has an associated value, and we wish to find a feasible solution with the best value. For example, in a problem that we call SHORTEST-PATH,

we are given an undirected graph G and vertices u and , and we wish to find a path from u to that uses the fewest edges. In other words, SHORTEST-PATH is the single-pair shortest-path problem in an unweighted, undirected graph. NP-completeness applies directly not to optimization problems, however, but to decision problems, in which the answer is simply “yes” or “no” (or, more formally, “1”or “0”).

The complexity class NP

The complexity class NP is the class of languages that can be verified by a poly-nomial-time algorithm.8 More precisely,